

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code: CSA06	Course Title: Design and Analysis of Algorithms
CO Assessed: CO1 – Precision (BL1, BL2, BL3)	Assessment: Assessment Tool 1 – Application Based Questions
Weightage: 40% of CIA	Total Marks: 100 (2 Questions × 50 Marks)

CO1 Statement: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)

Student Name: Dinesh Karthick R	Reg. No.: 192524121
Section / Batch:	Date: 17/07/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 – 50Q2 – 50

GRAND TOTAL: 100 Marks

SECTION A – APPLICATION BASED QUESTIONS

Q1. Digital Forensics Analysis System

A digital forensics system examines evidence files using the recurrence $T(n) = 8T(n/4) + n^2$. Apply the Master Theorem to determine the time complexity. Clearly identify parameters and justify the selected case. Analyze how performance changes as the volume of evidence increases. Interpret efficiency using asymptotic notation.

Q2. Smart Parking Management System

A smart parking system checks each arriving vehicle one by one to assign an empty slot. Identify the concepts of time complexity and space complexity, explain how the number of operations and memory usage grow with input size, and determine the asymptotic complexities of this process.

Code of Conduct

I, Dinesh Karthick R (Reg. No. 192524121), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Dinesh Karthick R

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q2. Smart Parking Management System – Answer (50 Marks)

1. Problem Definition

Here, every time a vehicle comes in, the system just checks the parking slots one by one to find an empty one. There are n slots total, and whether each one is empty or occupied is stored in an array of size n . So basically for each vehicle, it goes through this array step by step (this is called linear search) until it finds a free slot and assigns it.

2. Time Complexity Analysis

Best case is $O(1)$, this happens when the very first slot checked itself is free. Worst case is $O(n)$, this happens when only the last slot is free, or when no slot is free at all and the system has to check everything. Average case is also $O(n)$, since on average the free slot is found somewhere around the $n/2$ th check.

Basically if we increase the number of slots, the number of checks required also goes up the same way, like if slots get doubled, worst case checks also get doubled. That's why it's linear time, $O(n)$. (Also if m vehicles keep coming one after another, total time for all of them becomes $O(m \times n)$, but for just 1 vehicle it's still $O(n)$ only.)

3. Space Complexity Analysis

The array that stores whether each slot is empty/occupied takes up $O(n)$ space, since it has n entries.

Apart from that we only use a few counters and temp variables while scanning, which is just $O(1)$, so it's not really significant.

So the total space complexity is $O(n)$ overall, because memory needed grows along with the number of slots we're keeping track of.

4. Asymptotic Notation Summary

Big- O (upper bound) applies to the worst case and gives $O(n)$. Big- Ω (lower bound) applies to the best case and gives $\Omega(1)$. Big- Θ (tight bound) applies to the average case and gives $\Theta(n)$.

5. Conclusion

So overall, the Smart Parking Management System has $O(n)$ time complexity and $O(n)$ space complexity, because both the checking process and the memory needed to keep track of the slots increase directly with n , the number of parking slots.

Q1. Digital Forensics Analysis System – Answer (50 Marks)

1. Problem Understanding

In this system, the evidence set of size n is broken down into smaller parts, each part gets analyzed recursively, and then finally all the results are combined together. We can write this whole behaviour as a recurrence relation like this:

$$T(n) = 8T(n/4) + n^2$$

This is basically in the same form as the general Master Theorem equation $T(n) = aT(n/b) + f(n)$. Here the problem of size n gets split into a sub-problems, each one of size n/b , and $f(n)$ is the extra work done for dividing and then combining/merging the results back (like merging the evidence pieces together).

2. Identifying Parameters

Here $a = 8$, which is the number of sub-problems or recursive calls at each level. $b = 4$, which is the factor by which the evidence size shrinks every time. And $f(n) = n^2$, which is the cost of dividing the evidence and then combining/merging the results back together.

3. Applying the Master Theorem

Step 1: First find the critical exponent, $n^{\log_b a}$:

$$\log_8 8 = \log 8 / \log 4 = 3/2 \Rightarrow n^{\log_b a} = n^{1.5}$$

Step 2: Now compare $f(n) = n^2$ with $n^{1.5}$

n^2 is clearly growing faster than $n^{1.5}$ (we can write $n^2 = \Omega(n^{1.5+\epsilon})$ taking $\epsilon = 0.5 > 0$), so this comes under Case 3 of the Master Theorem.

Step 3: We also need to check the regularity condition for Case 3, i.e. $a \cdot f(n/b) \leq c \cdot f(n)$ for some $c < 1$:

$$a \cdot f(n/b) = 8 \times (n/4)^2 = 8 \times n^2/16 = n^2/2 = 0.5 \cdot f(n)$$

Since $0.5 < 1$, this condition is satisfied, so Case 3 is confirmed.

4. Result

So from Case 3, $T(n) = \Theta(f(n))$, which gives us:

$$T(n) = \Theta(n^2)$$

5. Growth & Performance Analysis

As n (the evidence size) keeps growing, it's actually the $f(n) = n^2$ part, i.e. the divide-and-combine step, that decides how the system performs, not the recursive branching part (the 8 calls of size $n/4$ each). Since $f(n)$ already outweighs whatever the recursion tree contributes, the total running time basically comes from this top-level combining step alone. In simple words, if we double the amount of evidence, the processing time roughly becomes 4 times more, so this system won't scale well for really large evidence volumes. If we want to speed it up, the best thing to focus on would be optimizing that merge/combine part (the n^2 term).

6. Asymptotic Interpretation

Big-O (upper bound) gives $O(n^2)$. Big-Ω (lower bound) gives $\Omega(n^2)$. Big-Θ (tight bound) gives $\Theta(n^2)$.

Conclusion: So overall, this digital forensics system runs in $\Theta(n^2)$ time, which is quadratic complexity, because as shown by Case 3 of the Master Theorem, the combining step $f(n) = n^2$ is what dominates over the recursive part.